Middleware-i sıfırdan başa düşək

Brauzerdə bir ünvan açırsan: məsələn, `/hello`. Brauzer serverə sorğu göndərir. **Server** həmin sorğunu qəbul edib cavab hazırlayan proqramdır. **Sorğu - request** nə istədiyini bildirir. **Cavab - response** serverin geri göndərdiyi nəticədir. HTTP bu ünsiyyətin qaydalarıdır.

Ən sadə halda serverdə bir kod işləyir və sənə Salam qaytarır:

Sorğunun yolu

```
Brauzer -> Salam hazirlayan kod -> Brauzere cavab
```

İndi istəyirsən ki, Salam hazırlanmamışdan əvvəl sorğunun gəldiyi qeydə alınsın. Sonra da işin bitdiyi qeydə alınsın. Bunu cavab hazırlayan kodun ətrafına ayrıca mərhələ kimi yerləşdirə bilərsən:

Bir mərhələ əlavə etdik

```
Sorgu geldi
  -> Geldiyini qeyd et
  -> Salam cavabini hazirla
  -> Bitdiyini qeyd et
  -> Cavab
```

Middleware məhz belə bir mərhələdir. Sorğuya baxır, lazım olan işi görür, sonra davamına keçə bilər. Bəzən davamı işlətmədən özü cavab verir.

Belə mərhələlərin bir-birinə qoşulduğu yola **pipeline** deyirik. Burada pipeline sorğunun keçdiyi mərhələlər ardıcıllığıdır.

Bir azdan yalnız bir middleware yazacağıq. Hələ istifadəçi yoxlaması, DI və başqa mövzular barədə düşünməyə ehtiyac yoxdur. Əvvəl bu fikri saxla: **sorğunu işləyən əsas koddan əvvəl və sonra ayrıca iş görə bilərik.**

Bir middleware yazıb nəticəsinə baxaq

Bu, boş ASP.NET Core layihəsinin Program.cs faylına yazıla bilən tam nümunədir. Terminalda `dotnet new web -n MiddlewareDemo` ilə layihə yarada, həmin qovluqda `dotnet run` ilə işlədə bilərsən. Terminalda göstərilən ünvana /hello əlavə edib aç.

C# - Program.cs

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.Use(async (context, next) =>
{
    Console.WriteLine("A: evvel");
    await next(context);
    Console.WriteLine("A: sonra");
});

app.MapGet("/hello", () =>
{
    Console.WriteLine("Salam hazirlanir");
    return "Salam";
});

app.Run();
```

İlk iki sətir tətbiqi hazırlayır. **Use** bizim mərhələni əlavə edir; ona A adı vermişik. **MapGet** /hello ünvanına GET sorğusu gələndə işləyəcək əsas kodu qeyd edir. GET burada məlumat istəmək üçün sorğu növüdür. Bu konkret işləyici koda **endpoint** deyirik.

context cari sorğu və cavabı saxlayan obyektdir. **next** isə səndən sonrakı hissəni işlətmək üçün çağırdığın funksiyadır. `await next(context)` həmin hissəni işlədir, bitməsini gözləyir, sonra davam edir. await asinxron işin tamamlanmasını gözləmək üçün yazılır; async belə gözləmə olan funksiyanı işarələyir.

Brauzer Salam görür. Terminalda isə bunlar yazılır:

Bizim qeydlərimiz

```
A: evvel
Salam hazirlanir
A: sonra
```

Sorğunu qeydə alan kodla Salam qaytaran kod ayrı qaldı. Middleware-in faydası da belə ümumi işləri hər endpoint-in içində təkrarlamamaqdır.

next-dən sonra kod niyə geri sıra ilə işləyir?

Əvvəlki proqramda A middleware-indən sonra, MapGet-dən əvvəl ikinci mərhələ əlavə et:

C# - əvvəlki proqrama əlavə edilən B middleware-i

```
app.Use(async (context, next) =>
{
    Console.WriteLine("B: evvel");
    await next(context);
    Console.WriteLine("B: sonra");
});
```

İndi proqramda sıra A, B, sonra /hello işləyicisidir. /hello sorğusunun çıxışı:

Bizim qeydlərimiz

```
A: evvel
B: evvel
Salam hazirlanir
B: sonra
A: sonra
```

Sətir-sətir nə baş verdi?

- A əvvəl öz işini gördü. next çağırıb B-ni gözlədi.

- B öz işini gördü. next çağırıb ondan sonrakı hissəni gözlədi.

- /hello işləyicisi Salam cavabını hazırladı və bitdi.

- B-nin gözlədiyi iş bitdiyi üçün B sonra sətri işlədi.

- B bitdiyi üçün A da gözləmədən çıxdı və A sonra sətri işlədi.

A və B yenidən əvvəldən çağırılmır. Onlar gözlədikləri çağırış bitəndə növbəti sətirdən davam edirlər. Bu, bir funksiyanın başqa funksiyanı çağırıb sonra öz işinə davam etməsi kimidir.

Müsahibədə bunu soruşsalar: **“Sorğu daxil olanda qeydiyyat sırası ilə, next-dən sonrakı hissələr isə normal tamamlanmada əks sıra ilə işləyir”** deyə bilərsən. Xəta ilə çıxışın fərqinə son hissədə baxacağıq.

next çağırmasaq nə baş verər?

Tutaq ki, /blocked yoluna gələn sorğunu irəli buraxmaq istəmirik. B middleware-ində next-dən əvvəl bu yoxlamanı qoyuruq:

C# - B middleware-ini bununla əvəz et

```
app.Use(async (context, next) =>
{
    Console.WriteLine("B: evvel");

    if (context.Request.Path == "/blocked")
    {
        context.Response.StatusCode = 403;
        await context.Response.WriteAsync("Icaze yoxdur");
        return;
    }

    await next(context);
    Console.WriteLine("B: sonra");
});
```

Request.Path gələn ünvanın yol hissəsidir. **Response.StatusCode** cavabın nəticə kodudur; 403 bu nümunədə girişə icazə verilmədiyini bildirir. **WriteAsync** məzmunu cavaba yazır. **return** cari funksiyadan çıxır.

/blocked gələndə B özü cavab hazırlayır və next-ə çatmır. Ondan sonrakı hissə işləməyəcək. Buna **short-circuit**, yəni davamı kəsmək deyilir.

/blocked üçün qeydlər

```
A: evvel
B: evvel
A: sonra
```

Niyə A sonra var? Çünki A artıq B-ni çağırmışdı. B normal şəkildə bitəndə A gözlədiyi çağırışdan geri qayıdıb davam edir. **Dayanan hissə B-dən sonrakı yoldur; A-nın qalan kodu silinmir.**

/hello gəlsə if şərti ödənmir. B next-i çağırır və əvvəl gördüyümüz tam sıra işləyir. Real layihədə hazır faylı cavablandıran və ya icazəsiz sorğunu saxlayan mərhələ də davamı bu şəkildə kəsə bilər.

Real tətbiqdə mərhələləri niyə bu sırada qoyuruq?

İndi sifarişləri göstərən /orders sorğusunu düşün. Əsas işləyici sifariş siyahısını hazırlayır. Ondan əvvəl ümumi yoxlamalar ola bilər:

Məntiqi yol

```
Xetani tutan hisse
  -> Hansi endpoint-dir?
  -> Istifadeci kimdir?
  -> Buraya girmeye icazesi var?
  -> Sifaris siyahisini hazirla
```

Routing “bu ünvanı hansı işləyici cavablandırmalıdır?” seçimini edir. **Authentication** istifadəçinin kimliyini müəyyənləşdirir. **Authorization** həmin istifadəçinin seçilmiş əməliyyata icazəsini yoxlayır.

Məsələn, istifadəçi tanınmış ola bilər, amma bütün sifarişləri görmək icazəsi olmaya bilər. Buna görə kimliyi müəyyənləşdirmək və icazəni yoxlamaq ayrı işlərdir.

Sıra nəticəyə təsir edir. Kimlik barədə məlumatdan istifadə edən icazə yoxlamasından əvvəl istifadəçini tanımaq lazımdır. Xəta tutan hissə də davamını çağırarkən baş verən xətanı tuta bildiyi üçün əvvəl yerləşir.

Bu komponentlər hər biri işi yenidən controller-in içində yazmamağa kömək edir. **Controller** sorğuları işlədən metodları bir yerə toplayan sinifdir. Onun konkret işləyici metoduna **action** deyilir.

ASP.NET Core bəzi mərhələləri uyğun ayarlara əsasən avtomatik qoşa bilir. Burada göstərilən məntiqi sıradır; hər proqramda bütün metodları əl ilə yazmalısan mənasına gəlmir.

Endpoint qeydiyyatı da onu dərhal işlətmir. `app.MapGet(...)` “belə sorğu gələndə bu kodu işlət” qaydasını saxlayır. Sorğu gələndə seçilmiş endpoint-ə çatılarsa həmin kod icra olunur.

Use, Run və Map adlarını qarışdırmayaq

Use: mərhələ əlavə et. Əvvəlki nümunələrdəki kimi next ilə davam edə, ya da özü cavab verə bilər.

Run(context => ...): burada cavablandır. next parametri almayan sonlandırıcı işləyicidir:

C# - ayrıca kiçik proqramın işləyicisi ola bilər

```
app.Run(async context =>
{
    await context.Response.WriteAsync("Salam");
});
```

Bunun arxasındakı əsas pipeline mərhələlərinə davam edilmir. Amma **parametrsiz app.Run()** başqa çağırışdır: tətbiqi işə salır. Söz eyni olsa da istifadə formaları və vəzifələri fərqlidir.

Map: müəyyən yola ayrıca qol ayır. Məsələn, /admin ilə başlayan uyğun yollar üçün ayrıca işləmə yolu qura bilərsən:

C# - əsas proqrama qol əlavə etmək nümunəsi

```
app.Map("/admin", branch =>
{
    branch.Run(async context =>
    {
        await context.Response.WriteAsync("Admin bolmesi");
    });
});
```

branch həmin ayrıca qolun qurucusudur. Uyğun sorğu bu qola keçir, sonra əsas pipeline-ın arxadakı hissəsinə birləşmir. Əvvəldəki middleware-lərin gözləyən kodu yenə geri davam edə bilər.

UseWhen şərtə əsasən qol yaratmaq üçündür. Qol cavab verib davamı kəsmirsə əsas yola birləşə bilər. **MapGet** isə əvvəl gördüyün konkret GET endpoint-inin qeydiyyatıdır; yuxarıdakı Map ilə eyni quruluş deyil.

Yadda saxlamaq üçün: **Use davam edə bilən mərhələ, Run işləyicisi sonlandırma, Map ayrıca yoldur.** Parametrsiz Run isə proqramı işə salır.

Siniflə yazılan middleware və onun ömrü

İndiyədək middleware-i birbaşa Program.cs-də yazdıq. Eyni mərhələni ayrıca sinifdə də saxlaya bilərik. Aşağıdakı sinif gələn yolu qeyd edir və davamını çağırır:

C# - RequestLogMiddleware.cs

```
public class RequestLogMiddleware
{
    private readonly RequestDelegate _next;

    public RequestLogMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        Console.WriteLine(context.Request.Path);
        await _next(context);
    }
}
```

RequestDelegate əvvəlki next funksiyasının tipidir. **HttpContext** əvvəlki context obyektinin tipidir. **Task** tamamlanması gözlənilən işi təmsil edir. **InvokeAsync** bu mərhələyə çatan sorğunu işləyən metoddur.

Program.cs-də əvvəlki mərhələni bu siniflə əvəz etmək üçün `app.UseMiddleware<RequestLogMiddleware>();` yazırıq.

Bu adi yanaşmada sinif pipeline qurularkən yaradılır. Hər sorğuda InvokeAsync çağırılır, amma middleware obyektinin özü hər sorğuda yenidən yaranmır. Bir neçə sorğu eyni obyektə eyni vaxtda çata bilər. Ona görə cari istifadəçinin adını sinfin dəyişənində saxlamaq düzgün deyil; sorğuya aid məlumatı həmin çağırışın context-ində və ya lokal dəyişənində saxla.

DI obyektin lazım olan servisini kənardan almasıdır. **Scoped servis** bir iş sərhədində paylaşılan obyektdir; adi HTTP sorğusunda həmin sərhəd cari sorğudur. Belə servisi adi middleware-in konstruktoruna deyil, InvokeAsync-in əlavə parametrinə alırlar. **IMiddleware** adlı başqa yaradılma üsulunda isə scoped servis konstruktorda da alına bilər. Bu səbəbdən bütün middleware obyektlərinin ömrünü eyniləşdirmə.

İndi müsahibə suallarını öz sözlərinlə cavablandır

Middleware nədir? HTTP sorğusu və cavabı üzərində işləyən mərhələdir. Davamından əvvəl və sonra iş görə, yaxud davamı kəsib özü cavab verə bilər. Pipeline bu mərhələlərin ardıcıllığıdır.

next nə edir? Qalan hissəni çağırır. await ilə gözləyəndə həmin hissə bitdikdən sonra indiki funksiyanın növbəti sətri işləyir.

next-dən sonrakı kod həmişə işləyir? Xeyr. Davamda tutulmayan xəta yuxarı atılarsa adi sətirə keçilməyə bilər. Həm uğurlu, həm xətalı çıxışda iş görmək üçün try/finally istifadə olunur. finally blokdan çıxış işini görür; xətanı özü tutmur. Xətanı tutmaq üçün catch və ya uyğun xəta middleware-i lazımdır.

Cavabı next-dən sonra dəyişə bilərəm? Ehtiyatla: cavab artıq göndərilməyə başlayıbsa onun statusunu və başlıqlarını dəyişmək mümkün olmaya bilər. Başlıq cavabın Content-Type kimi əlavə məlumatıdır. Response.HasStarted başlanıb-başlanmadığını bildirir.

Middleware ilə filter fərqi? Middleware ümumi HTTP yolundadır. MVC filter controller action-ının icra mərhələlərinə qoşulur; məsələn, onun arqumentlərini yoxlamaq üçün işlənə bilər. Action-a çatılmasa həmin filter də işləməyə bilər.

Hər middleware hər sorğuda işləyir? Yalnız həmin hissəyə çatılırsa. Əvvəlki mərhələ dayandıra və ya sorğu ayrıca qola gedə bilər. Uyğun endpoint tapılmır və başqa cavab verən hissə də yoxdursa adətən 404 qaytarılır. Bu, mütləq proqram xətası atılması deyil.

Hansı pattern-ə oxşayır? Chain of Responsibility: mərhələ işi görüb davamı ötürür və ya yolu kəsir. Bu, quruluşun dizayn adıdır; işləmə məntiqini isə A və B nümunəsində gördün.

Rəsmi istinadlar: əsas middleware davranışı, siniflə yazılış və DI, IMiddleware, HttpContext, MVC filters.